

## Module 11 - chapter 9

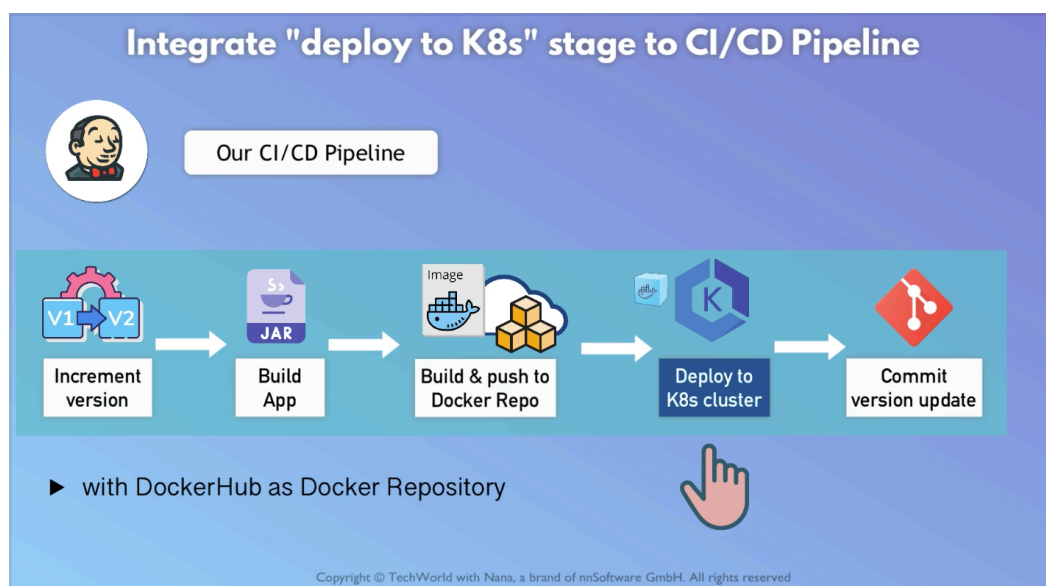
### Complete CI/CD Pipeline with EKS and DockerHub

#### Part 1

In this lecture we are working on the "jenkins-jobs" branch of the java-maven-app project, which can be found here: <https://gitlab.com/twn-devops-bootcamp/latest/11-eks/java-maven-app/-/tree/jenkins-jobs>

The starting code can be found here: <https://gitlab.com/twn-devops-bootcamp/latest/11-eks/java-maven-app/-/tree/starting-code>

Integrating the 'deploy to k8s' stage to CI/CD Pipeline



This is the part that we want to integrate into our existing pipeline:

```

15     script {
16         echo "building the docker image..."
17     }
18 }
19
20 stage('deploy') {
21     environment {
22         AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_key_id')
23         AWS_SECRET_ACCESS_KEY = credentials('jenkins_aws_secret_access_key')
24     }
25     steps {
26         script {
27             echo 'deploying docker image...'
28             sh 'kubectl create deployment nginx-deployment --image=nginx'
29         }
30     }
31 }
32
33 }

```

project: "java-maven-app"

feature branch: "deploy-on-k8s"

Terminal Services Build Dependencies

TECHWORLD WITH NANA

And we are going to copy the above and paste it into our new branch in the Jenkinsfile (this Jenkins file can be refactored later using the shared-library, etc but for learning we will have all the code here)

## Current Jenkinsfile in new branch

Increment version stage:

```

8     stages {
9         stage('increment version') {
10             steps {
11                 script {
12                     echo 'incrementing app version...'
13                     sh 'mvn build-helper:parse-version versions:set \
14                         -DnewVersion=\\${parsedVersion.majorVersion}.\\${parsedVersion.minorVersion}.\\${parsedVersion.nextIncrementalVersion} \
15                         versions:commit'
16                     def matcher = readFile('pom.xml') =~ '<version>(.)+</version>'
17                     def version = matcher[0][1]
18                     env.IMAGE_NAME = "$version-$BUILD_NUMBER"
19                 }
20             }
21         }
22         stage('build app') {
23             steps {
24                 script {
25                     echo 'building app...'
26                     sh 'mvn clean package'
27                 }
28             }
29         }
30     }

```

Increment version → Build App → Build & push to Docker Repo → Deploy to K8s cluster → Commit version update

Download pre-built shared indexes: Reduce the indexing time and

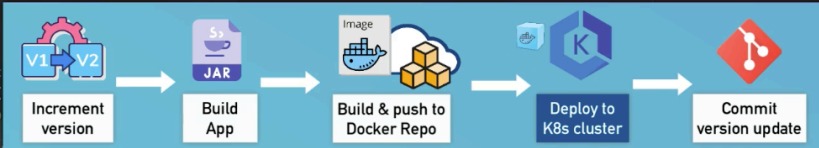
Build app stage:

We clean - remove the previous build and create a new jar

```

21 }
22 stage('build app') {
23     steps {
24         script {
25             echo 'building the application...'
26             sh 'mvn clean package'
27         }
28     }
29 }
30 stage('build image') {
31     steps {
32         script {
33             echo "building t
34             withCredentials(
35                 sh "docker b

```

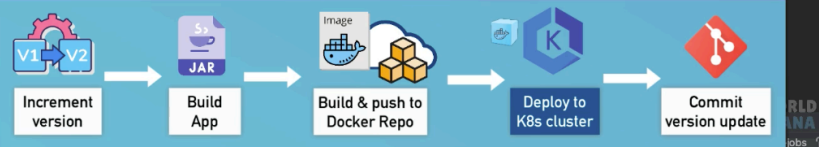
Build image stage:

```

30 stage('build image') {
31     steps {
32         script {
33             echo "building the docker image..."
34             withCredentials([usernamePassword(credentialsId: 'docker-hub-repo', passwordVariable: 'PASS', usernameVariable: 'USER')]){
35                 sh "docker build -t panatwn/demo-app:${IMAGE_NAME} ."
36                 sh 'echo $PASS | docker login -u $USER --password-stdin'
37                 sh "docker push panatwn/demo-app:${IMAGE_NAME}"
38             }
39         }
40     }
41 }
42 stage('deploy') {
43     steps {
44         script {
45             echo 'deploying d
46         }
47     }
48 }
49 stage('commit version update'){
50     steps {
51         script {
52             withCredentials(
53                 sh "git remo
54                 sh 'git add
55                 sh 'git comm
56                 sh 'git push

```





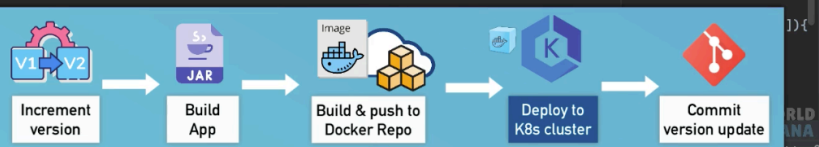
Deploy stage:

Currently empty

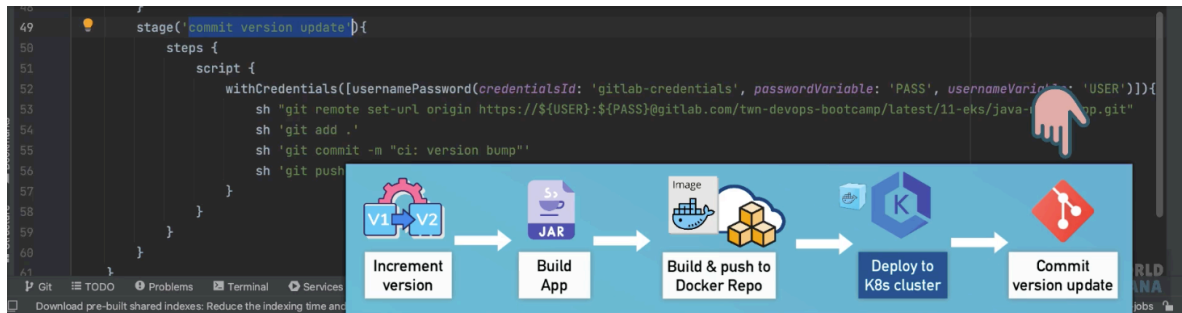
```

42 stage('deploy') {
43     steps {
44         script {
45             echo 'deploying docker image...'
46         }
47     }
48 }
49 stage('commit version update'){
50     steps {
51         script {
52             withCredentials(
53                 sh "git remo
54                 sh 'git add
55                 sh 'git comm
56                 sh 'git push

```

Commit the version increment to the git repo stage:



## Edit deploy stage

Copy code from Module 11 - chapter 6. To get the AWS credentials:

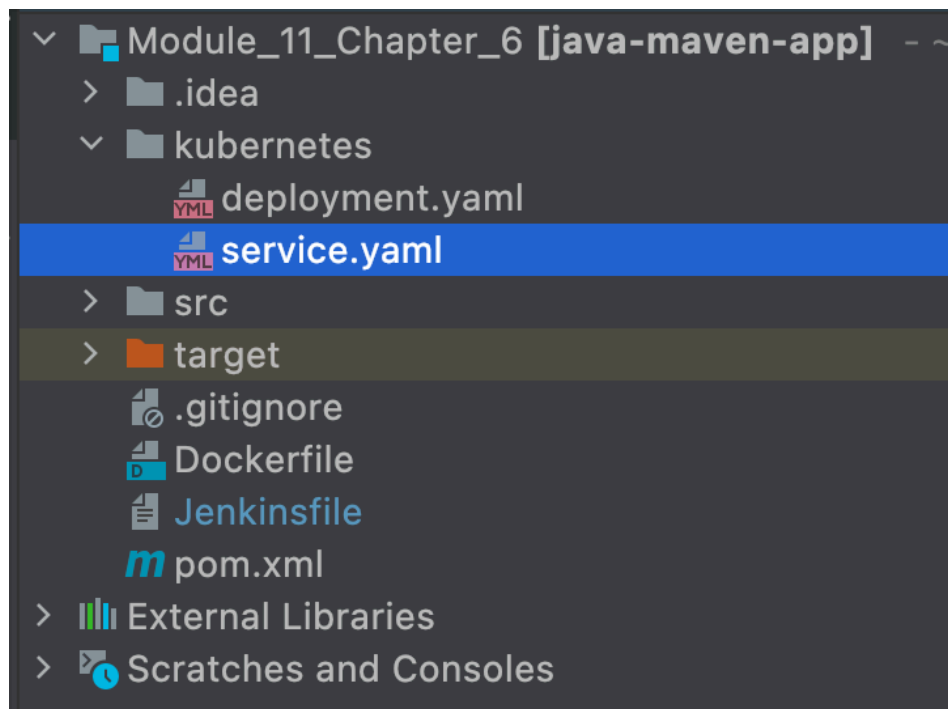
```
Jenkinsfile
40   }
41 }
42 stage('deploy') {
43   environment {
44     AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_key_id')
45     AWS_SECRET_ACCESS_KEY = credentials('jenkins-aws_secret_access_key')
46   }
47   steps {
48     script {
49       echo 'deploying docker image...'
50       sh 'kubectl create deployment nginx-deployment --image=nginx'
51     }
52   }
53 }
54 stage('commit version update'){
```

We will edit line 50 -> we want to deploy our IMAGE the one we build in 'build image' stage:

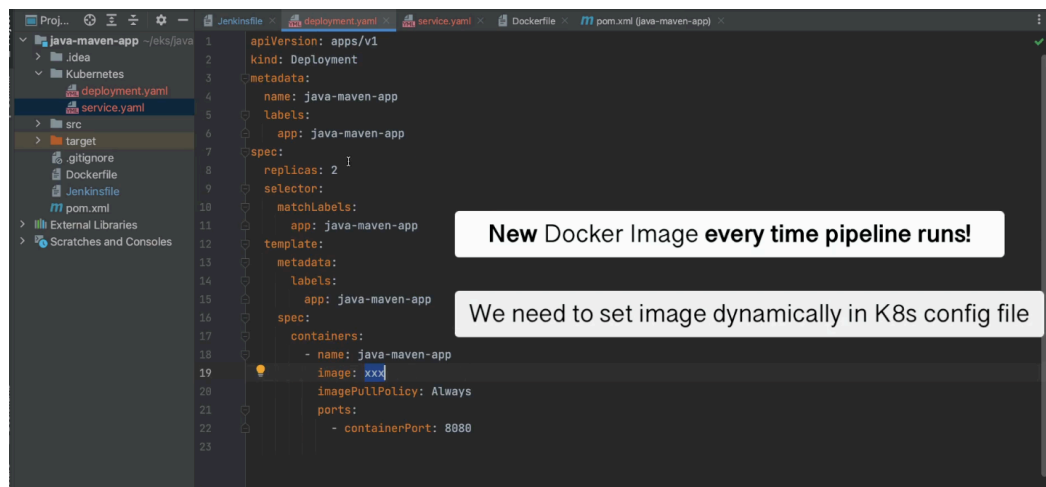
```
30 stage('build image') {
31   steps {
32     script {
33       echo "building the docker image..."
34       withCredentials([usernamePassword(credentialsId: 'docker-hub-repo', passwordVariable: 'PASS', usernameVariable: 'USER')]){
35         sh "docker build -t ${DOCKER_REPO}:${IMAGE_NAME} ."
36         sh 'echo $PASS | docker login -u $USER --password-stdin ${DOCKER_REPO_SERVER}'
37         sh "docker push ${DOCKER_REPO}:${IMAGE_NAME}"
38       }
39     }
40   }
41 }
```

**New Docker Image every time pipeline runs!**

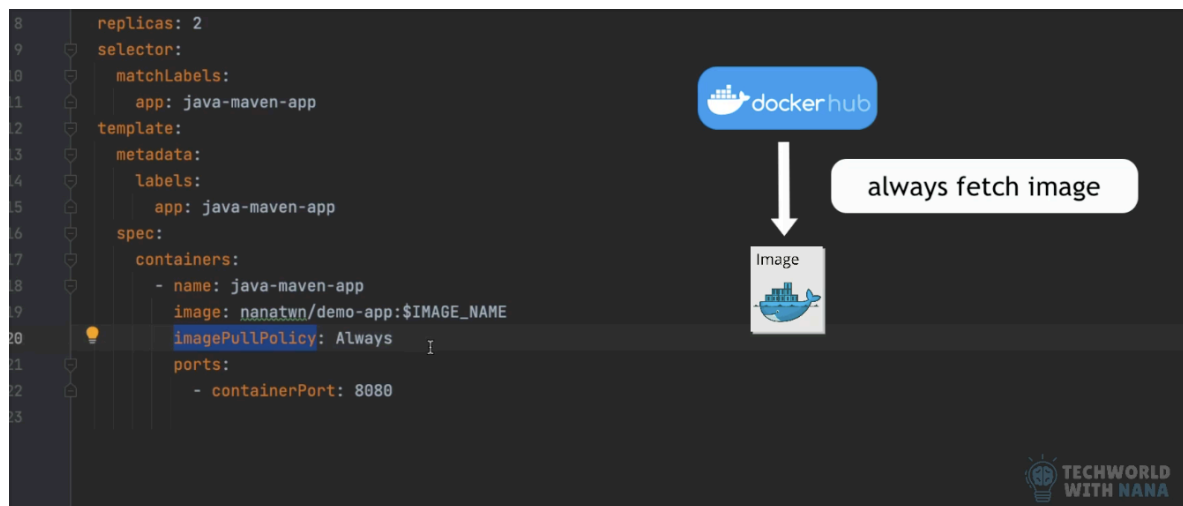
Create Deployment and Service and adjust Jenkins file



The image name is created dynamically in Jenkinsfile 'build image' stage and that needs to be reflected in the deployment.yaml container section image attribute:

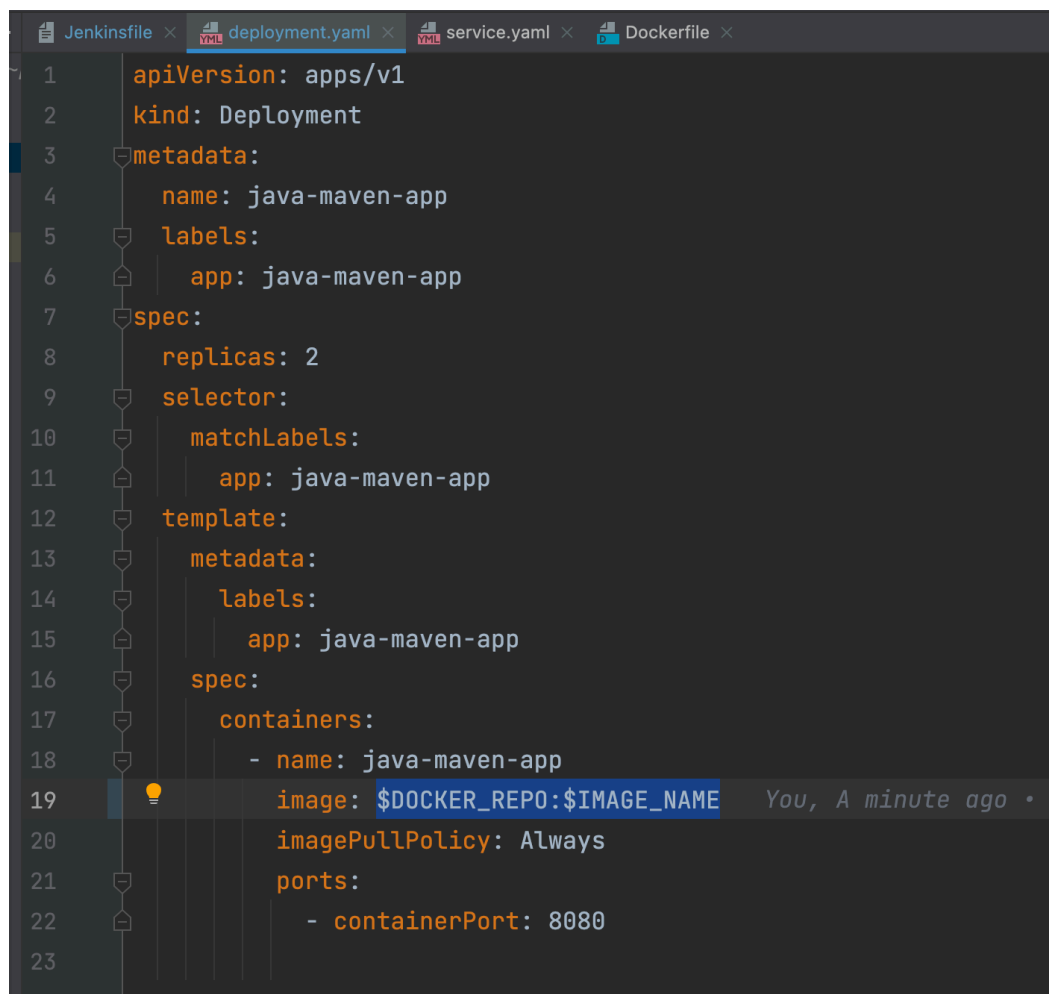


For that we will use the env var `$IMAGE_NAME` -> to set the image dynamically in K8s config file



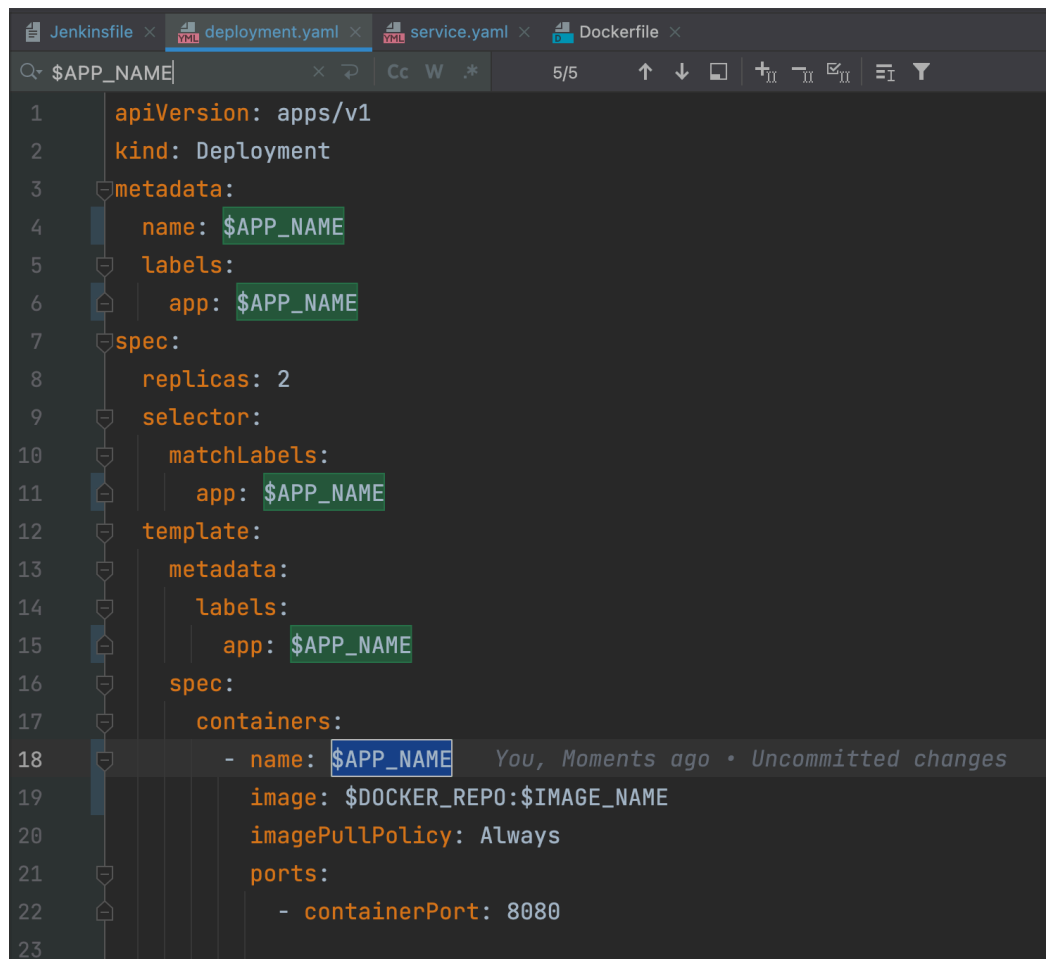
And we set the attribute 'imagePullPolicy' to 'Always' -> when the pod starts always fetch a new image.

Here is my deployment file so far:



We can also use an env var for the app name 'java-maven-app' as it is repeated in both deployment.yaml and service.yaml  
-> \$AAP\_NAME

Now the deployment file looks like:



```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: $APP_NAME
5    labels:
6      app: $APP_NAME
7  spec:
8    replicas: 2
9    selector:
10     matchLabels:
11       app: $APP_NAME
12    template:
13     metadata:
14       labels:
15         app: $APP_NAME
16     spec:
17       containers:
18       - name: $APP_NAME
19         image: $DOCKER_REPO:$IMAGE_NAME
20         imagePullPolicy: Always
21         ports:
22         - containerPort: 8080
23
```

And the service file:

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: $APP_NAME
5  spec:
6    selector:
7      app: $APP_NAME
8    ports:
9      - protocol: TCP
10        port: 80
11        targetPort: 8080
```

We need to create the env var \$AAP\_NAME in jenkinsfile. It can be set anywhere in the Jenkins file:

For example as a global var:

```
3  pipeline {
4    agent any
5    tools {
6      maven 'Maven'
7    }
8    environment {
9    }
10   stages {
```

Or I can use this pattern -> env.IMAGE\_NAME =  
And added to any stage:

```

8      stages {
9          stage('increment version') {
10              steps {
11                  script {
12                      echo 'incrementing app version...'
13                      sh 'mvn build-helper:parse-version versions:set \
14                        -DnewVersion=\\${parsedVersion.majorVersion}.\\${parsedVersion.minorVersion}.\\${
15                          versions:commit}'
16                      def matcher = readFile('pom.xml') =~ '<version>(.)</version>'
17                      def version = matcher[0][1]
18                      env.IMAGE_NAME = "$version-$BUILD_NUMBER" Janashia, 01/06/2023, 15:32 • Update Jenkins
19                  }
20              }

```

but we will do it in the deployment stage inside the environment block (so we don't need 'env.APP\_NAME')

```

41      }
42      stage('deploy') {
43          environment {
44              AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_key_id')
45              AWS_SECRET_ACCESS_KEY = credentials('jenkins_aws_secret_access_key')
46              APP_NAME = 'java-maven-app' You, Moments ago • Uncommitted changes
47          }
48          steps {

```

How are we passing the env vars from the Jenkinsfile to the yaml files?

We usually execute -> kubectl apply -f 'parameter'

But in this case the parameter is a file that is a template, it has the reference to variables \$APP\_NAME and \$IMAGE\_NAME so we need to use  
-> envsubst

envsubst -> environment substitute command, substitutes any variable inside a file

And envsubst is a tool that needs to be installed in the Jenkins container ⚠

```

47      steps {
48          script {
49              echo 'deploying docker image...'
50              sh 'envsubst kubectl apply -f '
51          }
52      }

```

"envsubst"

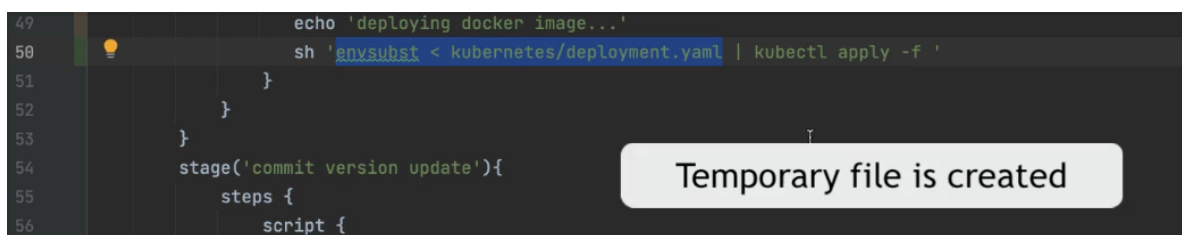
We need to pass the file with its relative path -> kubernetes/deployment.yaml



```
47 }
48 steps {
49   script {
50     echo 'deploying docker image...'
51     sh 'envsubst < kubernetes/deployment.yaml | > kubectl apply -f -'
52   }
53 }
```

And envsubst will look at the file and search for the \$ variables and match them with the env vars defined in that context (think like working in the terminal where each terminal tab is their own context, the session that jenkinsfile is running in is the context and jenkinsfile sets the env vars). So envsubst can see the env vars of the context and matches it with the \$ vars in the file. If there is a match then the file (in this case the yaml file) gets the values from the context. The file vars get substituted with the actual values.

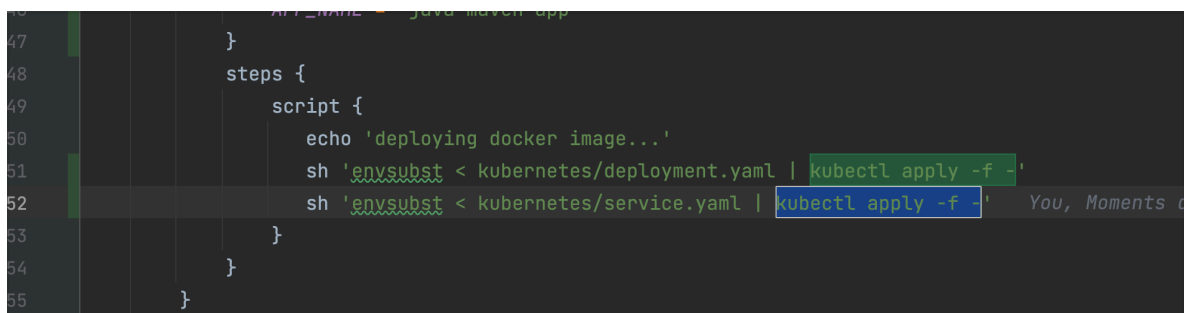
This part creates a temporary file, a file with the actual values (substituted) and then we pipe that temporary file with the kubectl apply command and pass this temporary file as a parameter:



```
49 echo 'deploying docker image...'
50 sh 'envsubst < kubernetes/deployment.yaml | kubectl apply -f -'
51 }
52 }
53 }
54 stage('commit version update'){
55   steps {
56     script {
```

Temporary file is created

And here is my jenkinsfile 'deploy' stage updated:



```
47 }
48 steps {
49   script {
50     echo 'deploying docker image...'
51     sh 'envsubst < kubernetes/deployment.yaml | kubectl apply -f -'
52     sh 'envsubst < kubernetes/service.yaml | kubectl apply -f -'
53   }
54 }
55 }
```

We need to install envsubst tool in our Jenkins container:

# Install "gettext-base" tool on Jenkins



Ssh into my droplet and Jenkins container as root user (to be able to install)

-> docker exec -u 0 -it ec090f856ef8 bash

-> apt-get update

-> apt-get install gettext-base

gettext-base -> package that includes envsubst tool

```
root@ec090f856ef8:/# apt-get install gettext-base
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Package gettext-base is not available, but is referred to by another package.
This may mean that the package is missing, has been obsoleted, or
is only available from another source

E: Package 'gettext-base' has no installation candidate
root@ec090f856ef8:/# apt-get update
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [212 kB]
Fetched 10.1 MB in 1s (8009 kB/s)
Reading package lists... Done
root@ec090f856ef8:/# apt-get install gettext-base
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  gettext-base
0 upgraded, 1 newly installed, 0 to remove and 39 not upgraded.
Need to get 243 kB of archives.
After this operation, 758 kB of additional disk space will be used.
Get:1 http://deb.debian.org/debian trixie/main amd64 gettext-base amd64 0.23.1-2 [243 kB]
Fetched 243 kB in 0s (5384 kB/s)
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. a
ntEnd/Dialog.pm line 79, <STDIN> line 1.)
debconf: falling back to frontend: Readline
Selecting previously unselected package gettext-base.
```

We type envsubst and nothing happens, that is good. It doesn't say 'not found'

```
root@ec090f856ef8:/# envsubst  
^[[^C  
root@ec090f856ef8:/#
```

#### NOTE:

I think I should add the installation of the gettext-base package to the dockerfile at some point, to create a customised Jenkins docker image with this included as well as including kubectl and aws-iam-authenticator, so when I run Jenkins docker image it is customised and it has the tools I need. For now my current container was created with the below parameters but if the container dies I will lose the newly installed gettext-base and if the droplet dies I will lose kubectl, kubeconfig file and aws-iam-authenticator 😞

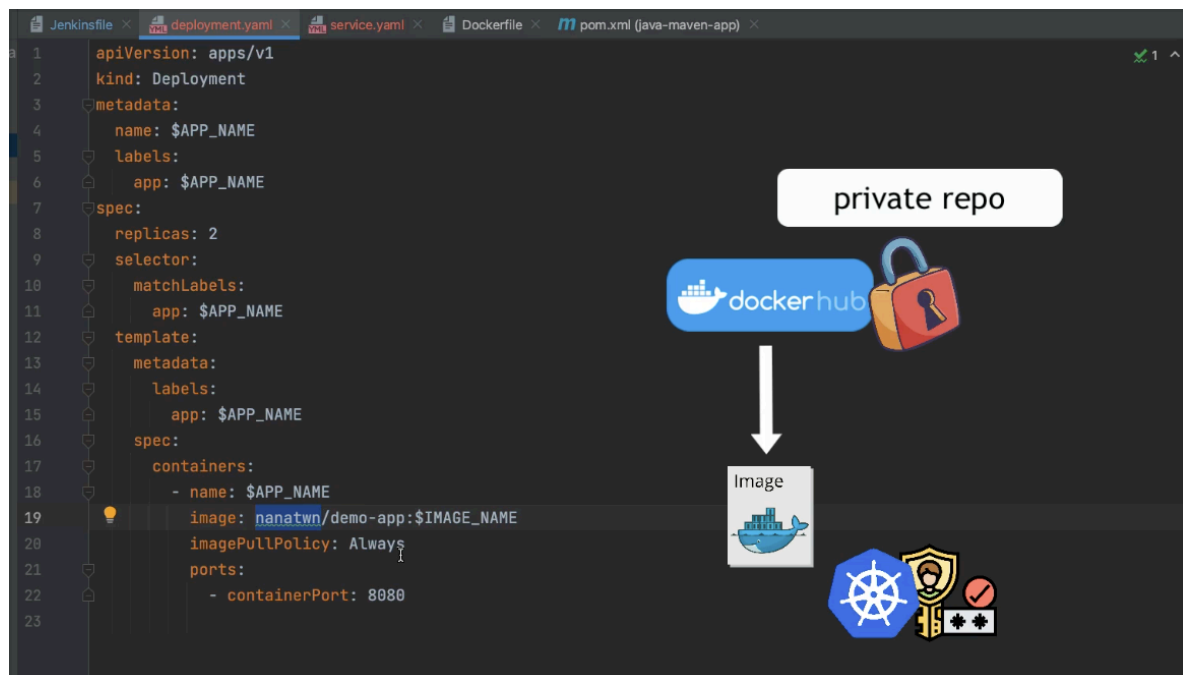
```
docker run -p 8080:8080 \  
-p 50000:50000 -d \  
-v /root/.kube:/var/jenkins_home/.kube \  
-v /usr/local/bin/kubectl:/usr/local/bin/kubectl \  
-v /usr/local/bin/aws-iam-authenticator:/usr/local/bin/aws-iam-authenticator \  
-v jenkins_home:/var/jenkins_home \  
jenkins/jenkins:its
```

### Create Secret for DockerHub credentials

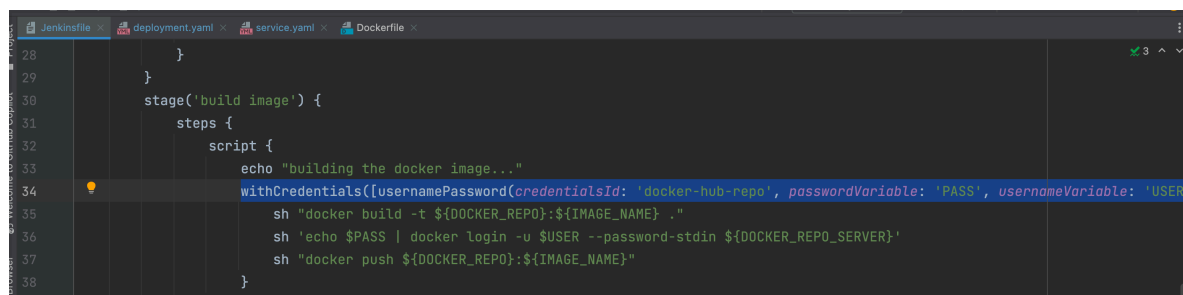


When 'deploy' stage from Jenkinsfile reaches the deployment.yaml file containers section and takes the image information it will then go to the dockerhub to pull the image.

For that k8s must be allowed to fetch the image from the private repository -> so k8s needs authentication with the registry

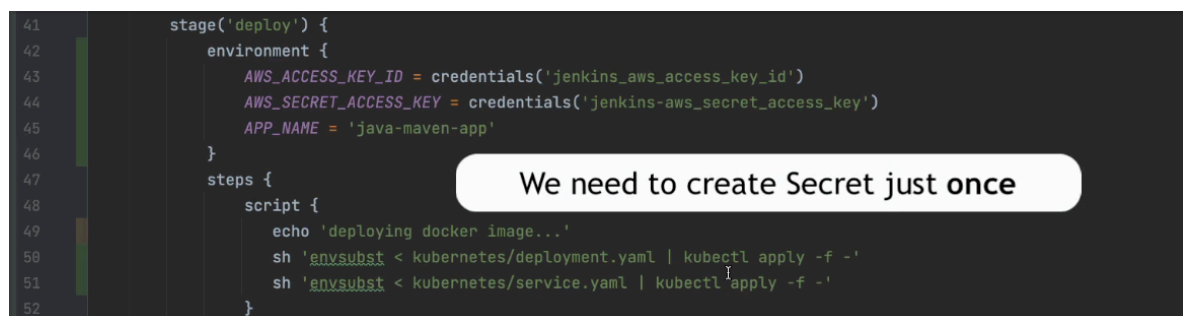


We will use the information from the 'build image' stage and put it in a secret.yaml file for k8s to use it



The secret is a one time event, is created once (unlike the deployment and the service) we don't need it to be created every single time Jenkins build is executed.

We could also add code to check whether the secret exists and create it if not, else don't create it using kubectl (as it is available in Jenkins container)



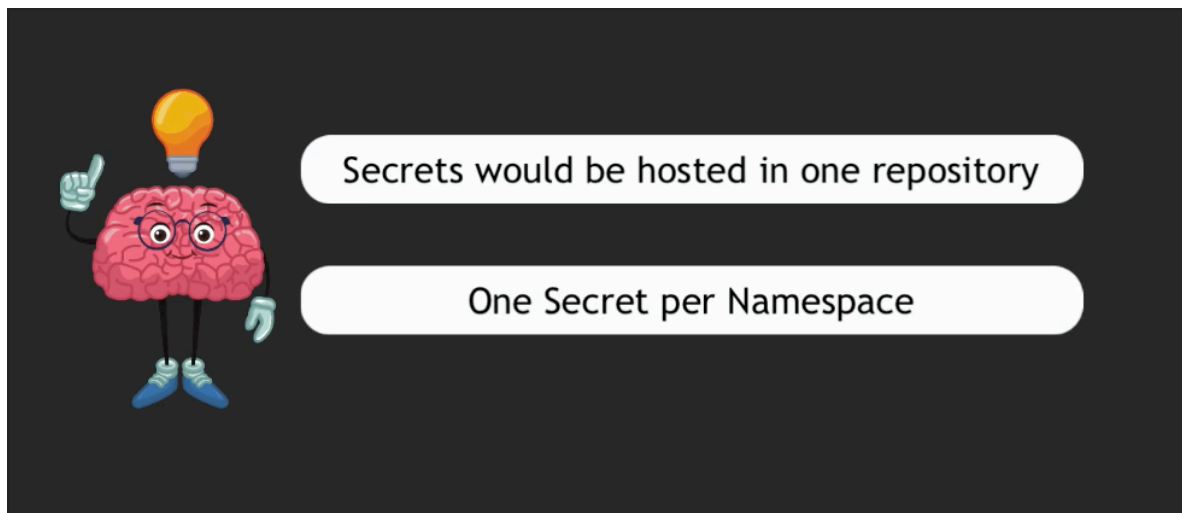
For this chapter we will create it from our local machine

But first I need to create my EKS cluster again (in a new terminal)

```
eksctl create cluster \
--name demo-cluster \
--version 1.35 \
--region eu-west-2 \
--nodegroup-name demo-nodes \
--node-type t2.micro \
--nodes 2 \
--nodes-min 1 \
--nodes-max 3
```

```
2026-06-11 11:14:05 [i] successfully created addon: metrics-server
2026-06-11 11:14:05 [i] kubectl command should work with "/Users/astrid/.kube/config", try 'kubectl get nodes'
2026-06-11 11:14:05 [✓] EKS cluster "demo-cluster" in "eu-west-2" region is ready
astrid@192.168.1.112 /Users/astrid
% kubectl get nodes
NAME                                STATUS    ROLES    AGE     VERSION
ip-192-168-56-185.eu-west-2.compute.internal Ready    <none>   2m23s   v1.35.5-eks-3385e9b
ip-192-168-86-86.eu-west-2.compute.internal Ready    <none>   2m29s   v1.35.5-eks-3385e9b
astrid@192.168.1.112 /Users/astrid
```

One secret per NS



Create the secret locally:

```
-> kubectl create secret docker-registry my-registry-key \
> --docker-server=docker.io \
> --docker-username=cabfish00 \
> --docker-password=*****
```

docker-registry -> type of secret

my-registry-key -> name for my secret

--docker-server=docker.io -> registry URL we want to authenticate with.

--docker-username=cabfish00 -> my dockerhub username

-docker-password=\*\*\*\*\* -> my dockerhub password

docker.io → DockerHub (Docker's official public registry)

When you pull nginx or jenkins/jenkins without specifying a registry, Docker implicitly uses docker.io.

---

For other registries:

|                                                           |          |
|-----------------------------------------------------------|----------|
| --docker-server=docker.io                                 | →        |
| DockerHub                                                 |          |
| --docker-server=<account>.dkr.ecr.eu-west-2.amazonaws.com | → AWS    |
| ECR                                                       |          |
| --docker-server=registry.gitlab.com                       | → GitLab |
| registry                                                  |          |
| --docker-server=ghcr.io                                   | → GitHub |
| Container Registry                                        |          |
| --docker-server=nexus.company.com:8083                    | → your   |
| Nexus registry                                            |          |

```
astrid@192.168.1.112 /Users/astrid
% secret/my-registry-key created

[1] + done      kubectl create secret docker-registry my-registry-key --docker-password=MpU
astrid@192.168.1.112 /Users/astrid
% kubectl get secret
NAME          TYPE          DATA   AGE
my-registry-key  kubernetes.io/dockerconfigjson  1       56s
astrid@192.168.1.112 /Users/astrid
% █
```

Now the secret is available and we can make use of it in our deployment.yaml file by adding:

imagePullSecrets:

- name: my-registry-key

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: $APP_NAME
5    labels:
6      app: $APP_NAME
7  spec:
8    replicas: 2
9    selector:
10     matchLabels:
11       app: $APP_NAME
12    template:
13     metadata:
14       labels:
15         app: $APP_NAME
16     spec:
17     imagePullSecrets:
18       - name: my-registry-key
19     containers:
20       - name: $APP_NAME
21         image: $DOCKER_REPO:$IMAGE_NAME
22         imagePullPolicy: Always
23         ports:
24           - containerPort: 8080
25
```

So every time k8s checks the deployment.yaml configuration it will have the secret available to use in order to pull the image.

I had to make other editions to my Jenkinsfile:

- Create two missing env vars -> I made them global variables

```
environment {
  DOCKER_REPO_SERVER = 'docker.io'
  DOCKER_REPO = 'cabfish00/demo-app'
}
```



Jenkins / EKS-chapter-11 / Configuration

### Configuration

- General
- Branch Sources
- Build Configuration
- Scan Repository Triggers
- Orphaned Item Strategy
- Appearance
- Health metrics
- Properties

Discover pull requests from forks ?

Strategy ?

The current pull request revision

Trust ?

From users with Admin or Write permission

May not be supported on older versions of GitHub Enterprise. See help button.

Filter by name (with regular expression)

Regular expression ?

Module\_11/EKS\_Full\_CI\_CD\_chapter-9

Add ▾

Property strategy

And

```

59     }
60     stage('commit version update'){
61         steps {
62             script {
63                 withCredentials([usernamePassword(credentialsId: 'github-credentials', passwordVari
64                     sh "git remote set-url origin https://${USER}:${PASS}@github.com/AstridCaballer
65                     sh 'git add .'
66                     sh 'git commit -m "ci: version bump"'
67                     sh 'git push origin HEAD:Module_11/EKS_Full_CI_CD_chapter-9' You, 14 minutes
68             }
69         }
70     }
71 }

```

## Execute Jenkins Pipeline

- push changes to GitHub

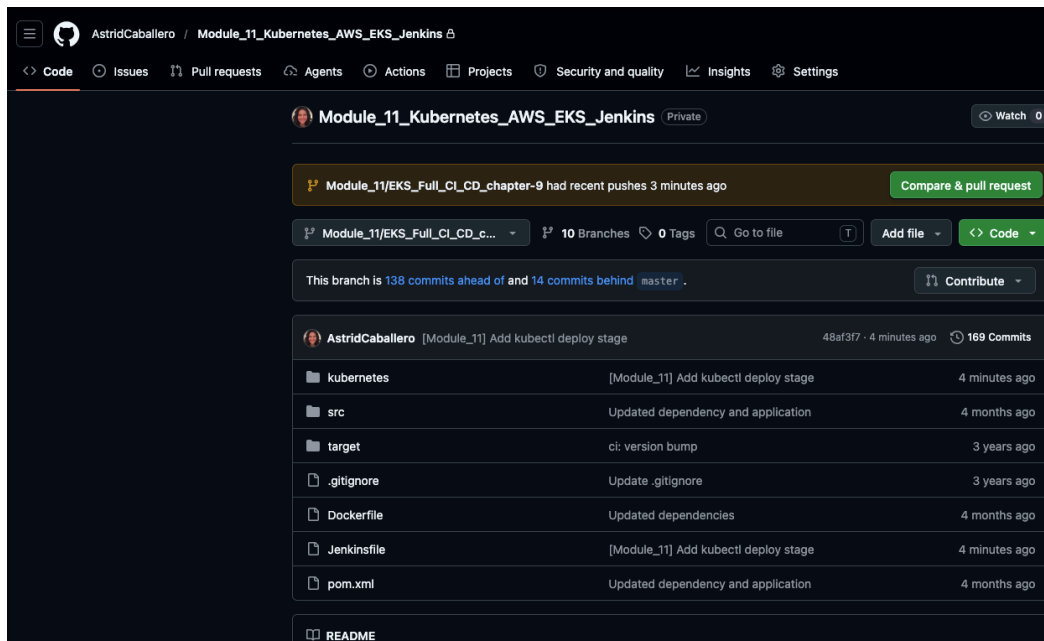
```

Terminal: Local + ▾
192:Module_11_Chapter_6 astrid$ git add .
192:Module_11_Chapter_6 astrid$ git commit -m "[Module_11] Add kubectrl deploy stage"
[Module_11/EKS_Full_CI_CD_chapter-9 48af3f7] [Module_11] Add kubectrl deploy stage
3 files changed, 24 insertions(+), 11 deletions(-)
192:Module_11_Chapter_6 astrid$ git push
fatal: The current branch Module_11/EKS_Full_CI_CD_chapter-9 has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin Module_11/EKS_Full_CI_CD_chapter-9

192:Module_11_Chapter_6 astrid$ git push --set-upstream origin Module_11/EKS_Full_CI_CD_chapter-9
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 1.33 KiB | 1.33 MiB/s, done.
Total 6 (delta 3), reused 1 (delta 1), pack-reused 8
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
remote:
remote: Create a pull request for 'Module_11/EKS_Full_CI_CD_chapter-9' on GitHub by visiting:
remote:   https://github.com/AstridCaballero/Module_11_Kubernetes_AWS_EKS_Jenkins/pull/new/Module_11/EKS_Full_CI_CD_chapter-9
remote:
To github.com:AstridCaballero/Module_11_Kubernetes_AWS_EKS_Jenkins.git
 * [new branch]      Module_11/EKS_Full_CI_CD_chapter-9 -> Module_11/EKS_Full_CI_CD_chapter-9
Branch 'Module_11/EKS_Full_CI_CD_chapter-9' set up to track remote branch 'Module_11/EKS_Full_CI_CD_chapter-9' from 'origin'.

```



- Check no pods are running in our EKS cluster  
-> kubectl get pod

```
astrid@192.168.1.112 /Users/astrid
% kubectl get pod
No resources found in default namespace.
astrid@192.168.1.112 /Users/astrid
% 
```

- Run the pipeline manually -> I haven't configured the Jenkins build to be executed when the GitHub repo is updated, my GitHub repo doesn't have a webhook yet also the jenkinsfile doesn't have the code to sign the commit/push when executing stage('commit version update')

```
java-maven-app Jenkinsfile
README.md pom.xml (java-maven-app) Jenkinsfile .gitignore Dockerfile
45         echo "deploying docker image"
46     }
47 }
48 }
49 stage('commit version update') {
50     steps {
51         script {
52             withCredentials([usernamePassword(credentialsId: 'github-credentials', passwordVariable: 'PASS', usernameVariable: 'U
53                 sh 'git config --global user.email "jenkins@example.com"'
54                 sh 'git config --global user.name "Jenkins"'
55
56                 sh 'git status'
57                 sh 'git branch'
58                 sh 'git config --list'
59
60                 sh "git remote set-url origin https://${USER}:${PASS}@github.com/AstridCaballero/java-maven-app.git"
61                 sh 'git add .'
62                 sh 'git commit -m "[ci skip] Increment version number"'
63                 sh 'git push origin HEAD:Module_8/chapter_17_multibranch'
64             }
65         }
66     }
67 }
```

\*See notes 'Multibranch Pipeline skip CI configuration' for more details

## First attempt it didn't work

because Jenkins file didn't specify the maven version:

```
Console Output
Branch indexing
11:32:10 Connecting to https://api.github.com using AstridCaballero/*****
Obtained Jenkinsfile from 48af3f7500710fee471316572ed2d1e9ed82d1f2
org.codehaus.groovy.control.MultipleCompilationErrorsException: startup failed:
WorkflowScript: 6: Tool type "maven" does not have an install of "Maven" configured - did you mean "maven-3.9"? @ line 6, column 15.
    maven 'Maven'
        ^
```

fixed:

```
3     pipeline {
4         agent any
5         tools {
6             maven 'maven-3.9'
7         }
8         environment {
9             DOCKER_REPO_SERVER = 'docker.io'
```

## Second attempt failed

because the container doesn't have docker:

```
[Pipeline] echo
building the docker image...
[Pipeline] withCredentials
Masking supported pattern matches of $PASS
[Pipeline] {
[Pipeline] sh
+ docker build -t cabfish00/demo-app:1.1.12-2 .
/var/jenkins_home/workspace/dule_11_EKS_Full_CI_CD_chapter-9@tmp/durable-d1e90926/script.sh.copy: 1: docker: not found
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
```

We need to give Jenkins container access to the droplet's docker daemon (see notes Module 8 - chapter 6)

The screenshot shows a Jenkins console output on the left and a diagram on the right. The console output includes the following commands and their results:

```
root@097251396f40:~# chmod 666 /var/run/docker.sock
root@097251396f40:~# ls -l /var/run/docker.sock
root@jenkins-server:~# ls -l /var/run/docker.sock
```

The diagram on the right illustrates the Docker architecture. It shows a 'Host/Server' at the bottom, which contains a 'docker runtime'. Inside the 'Host/Server', there is a 'Jenkins Container'. The 'Jenkins Container' also contains a 'docker runtime'. The diagram indicates that the Jenkins Container's docker runtime is connected to the Host/Server's docker runtime, allowing the Jenkins Container to access the Host/Server's docker daemon.

Below the diagram, the text states: "We will give Jenkins container access to our docker daemon (which is in the host the droplet). Then the container will be able to access docker."

**Tangent**  
 Note: not good practice to install docker inside the container:  
**Two ways to run Docker inside a container**  
 1. Docker-in-Docker (DinD)  
 • You install Docker *inside* the Jenkins container.  
 • Jenkins runs its own Docker daemon inside the container.  
 • Downsides:  
 • It's isolated from the host's Docker — images built inside don't appear on the host.

We need to mount docker into our container  
 -> -v /var/run/docker.sock:/var/run/docker.sock \

I stopped the previous container and I need to create a new Jenkins container:

```
docker run -p 8080:8080 \
  -p 50000:50000 -d \
  -v /root/.kube:/var/jenkins_home/.kube \
  -v /usr/local/bin/kubectl:/usr/local/bin/kubectl \
  -v /usr/local/bin/aws-iam-authenticator:/usr/local/bin/aws-iam-authenticator \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins/jenkins:its
```

```

root@jenkins-server:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
ec090f856ef8   jenkins/jenkins:lts   "/usr/bin/tini -- /u..." 44 hours ago   Up 27 hours   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0
:50000->50000/tcp, [::]:50000->50000/tcp   naughty_davinci
root@jenkins-server:~# docker stop ec090f856ef8
ec090f856ef8
root@jenkins-server:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
root@jenkins-server:~# docker run -p 8080:8080 \
-p 50000:50000 -d \
-v /root/.kube:/var/jenkins_home/.kube \
-v /usr/local/bin/kubectrl:/usr/local/bin/kubectrl \
-v /usr/local/bin/aws-iam-authenticator:/usr/local/bin/aws-iam-authenticator \
-v jenkins_home:/var/jenkins_home \
-v /var/run/docker.sock:/var/run/docker.sock \
jenkins/jenkins:lts
362b9015e4fa4728566554e99ad00ef55e35abe47c14206522e0724e314f112
root@jenkins-server:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
362b9015e4fa   jenkins/jenkins:lts   "/usr/bin/tini -- /u..." 4 seconds ago   Up 4 seconds   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0
.0:50000->50000/tcp, [::]:50000->50000/tcp   laughing_matsumoto
root@jenkins-server:~#

```

New container created!

### Third attempt failed

same reason -> docker not found.

I have connected the new container to docker daemon but I also need docker binary for docker CLI. So I am missing that.

Reminder from Module 8 - chapter 6:



### Docker Daemon (dockerd)

- **What it is:** The background service that actually does the work — building images, running containers, managing networks and volumes.
- **Role:** It's the "engine" of Docker.
- **Where it runs:** On the host machine (your droplet, for example).
- **How it works:** Listens for API requests (usually via the Unix socket / var/run/docker.sock) and executes them.
- **Example tasks:**
  - Create a container from an image
  - Build a new image from a Dockerfile
  - Manage container lifecycle (start, stop, remove)



### Docker CLI (docker)

- **What it is:** The command-line client you type commands into (docker run, docker ps, etc.).
- **Role:** It's just a **front-end tool** that sends requests to the daemon.
- **Where it runs:** Anywhere you install the Docker client binary (inside Jenkins, on your laptop, etc.).
- **How it works:** When you type docker run nginx, the CLI sends an API request to the daemon through the socket. The daemon then pulls the image and starts the container.
- **Example commands:**

- `docker ps` → asks the daemon for a list of running containers
- `docker build .` → tells the daemon to build an image from a Dockerfile

## **Analogy**

Think of Docker like a restaurant:

- **Docker CLI** = the waiter taking your order.
- **Docker Daemon** = the kitchen staff cooking the food.
- You don't need a kitchen inside every table (like installing Docker daemon inside Jenkins). You just need the waiter (CLI) to pass the order to the main kitchen (host daemon).

In Module 8 - chapter 6 I installed docker CLI directly in my Jenkins container, this time I will mount the droplet's docker CLI into my container 💡

```
-> -v /usr/bin/docker:/usr/bin/docker \
```

droplet has docker CLI at `/usr/bin/docker`

|  
mount it into the container

▼  
container can use it without installing anything

▼  
container restarts → still there (it's a mount not an installation)

Now I create. Anew Jenkins container with the new configuration included:

```
docker run -p 8080:8080 \
  -p 50000:50000 -d \
  -v /root/.kube:/var/jenkins_home/.kube \
  -v /usr/local/bin/kubect!:usr/local/bin/kubect! \
  -v /usr/local/bin/aws-iam-authenticator:/usr/local/bin/aws-iam-authenticator \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v /usr/bin/docker:/usr/bin/docker \
  jenkins/jenkins:its
```

```

root@jenkins-server:~# docker run -p 8080:8080 \
-p 50000:50000 -d \
-v /root/.kube:/var/jenkins_home/.kube \
-v /usr/local/bin/kubectl:/usr/local/bin/kubectl \
-v /usr/local/bin/aws-iam-authenticator:/usr/local/bin/aws-iam-authenticator \
-v jenkins_home:/var/jenkins_home \
-v /var/run/docker.sock:/var/run/docker.sock \
-v /usr/bin/docker:/usr/bin/docker \
jenkins/jenkins:lts
56efcedae10e58208177f2c082a3b682919a073c64f9b4c48b33dd252640607d
root@jenkins-server:~# docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS
56efcedae10e   jenkins/jenkins:lts  "/usr/bin/tini -- /u..." 3 seconds ago  Up 2 seconds  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp
naughty_dijkstra
root@jenkins-server:~#

```

New container created!

## Fourth attempt failed

Because I created a new container, the new container doesn't have the gettext-base package.

```

[Pipeline] echo
deploying docker image...
[Pipeline] sh
+ envsubst
/var/jenkins_home/workspace/dule_11_EKS_Full_CI_CD_chapter-9@tmp/durable-69a31a74/script.sh.copy: 1: envsubst: not found
+ kubectl apply -f -
error: no objects passed to apply
[Pipeline] }

```

So I need to install this in the Jenkins container:

-> docker exec -u 0 -it 56efcedae10e bash

-> apt-get update

-> apt-get install gettext-base

## Fifth attempt failed

Because the kubeconfig file has the configuration of an EKS that I deleted, today I created a new EKS so I need to update the config file with that information:

```

[Pipeline] script
[Pipeline] {
[Pipeline] echo
deploying docker image...
[Pipeline] sh
+ envsubst
+ kubectl apply -f -
error: error validating "STDIN": error validating data: failed to download openapi: Get "https://EBFD13B05DA0171EA96228C7F6E79CFD.yl4.eu-west-2.eks.amazonaws.com/openapi/v2?timeout=32s": dial tcp: lookup EBFD13B05DA0171EA96228C7F6E79CFD.yl4.eu-west-2.eks.amazonaws.com on 67.207.67.2:53: no such host; if you choose to ignore these errors, turn validation off with --validate=false
[Pipeline] }
[Pipeline] // script
[Pipeline] }

```

We get it from our local kubeconfig file, so I need to change the terminal to check my local file:

```

% cat .kube/config

```

-> cat .kube/config

And get the <certificate-data> and the <endpoint -url> to update the config that I currently have in my droplet:

```
apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority-data: <certificate-data>
    server: <endpoint-url>
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: aws
    name: aws
current-context: aws
users:
- name: aws
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      command: /usr/local/bin/aws-iam-authenticator
      args:
        - "token"
        - "-i"
        - <cluster-name>
```

```
~
~
~
~
~
~
~
~
~
~
```

```
:wq
```

Now it looks like this:

```

apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSB0tLS0tCk1JSURCVENDQWUyZ0F3SUJBZ0lJVUJyS3pUajVdZGt
    UTUJFR0ExVUUKQXhNS2EzVmIaWEp1WlhSbGNGQWVkdzB5TmBk1URXhNREF3TLROyUz3MhpOaEYtURneE1EQTF0VE5hTUJVeApFekFSQmdOVk
    1BMEddDU3FHU0l1M0RRRUJBUVVBQ0RJR3QXdnZ0VLCKFvSUJBURQcdnJzFRNHhRbTZyekU2bXJKRUhhWVhQdXVyVHBUUTVwU0xRa0N1VS9GT
    ThVdm5VUUnJ5RkRsd1VpWW0wN1hzU3h4cmF4YTY3ZF1YcENMSVFsSfc4LzZuWGs5Kz1uaWFDScgp2NXJFZk0R1F6SGNQa21DaENmSkpWcEQzUDN0
    MLN2dTNETmtrN2l1Ck12QVJ6SEJUMKxWVDkyN0xUS2cvYThPUlBld1IzOGVzZ2RHaWJCWLdHT291bDhEUST5cGo1R3dWMUVBSKxYM0EKd0JzVHQ
    LODJlU240QzdQazYrSHJkb2RDMWhhK1BJQmYrQXdTcUppdQp2Nz12NnF2ZXF1d0p2UDA4Zzd6cDZkeFayaUhsQWdNQkFBR2pXVEJYTUE0R0ExVW
    1CQWY4RUJUQURBUUgvTUIwR0ExVWREZ1FXQkJRQi90VDVzakJrUmM0cE1Zc09WM3VjU3Jjd11UQVYKQmdOVkhSRUVEakFNZ2dwcmlRSmxjbTVsZ
    TRJQkFRQU10d0swNTUwVWpFbnNBZ1YvRndXaUUSTmJpSERYNWh1bFo1SXQyeC9Ge1JMMytDOXdSeUVPMMXFKZmxSTlPldGFyQnd0UDhGNm12Cm83
    VzVvbH10eWFJVE5TSElPYjhtMTZVTvk83UGc0Z2Njbk1J4V1R3d0oKb0RrVT4ZGp2aW9pU0QrVTk1ekJ0Z282ej1obytmWWZSYTMwcmtyYb2N0WHJ
    EcW1VaU51bHNuQ3V1NnFNay8yeGwwWjRLN0QvYVpjMWSHZ3IrrYUZXN2pnzbz1ZlVTFaYk02T3Jnei92VUplClp1QmZFL3FVenJvRmVMSnZhSU1qVE
    Y1anYraGdQZkRubEYwTU16TXkkULRsWWcyOHhVZWt2Ci0tLS0tRU5EIENFUlRJRklDQVRFLS0tLS0K
    server: https://FD3BB1916E6CB04187F720B7D7C0EA1B.gr7.eu-west-2.eks.amazonaws.com
name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: aws
name: aws
current-context: aws
users:
- name: aws
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      command: /usr/local/bin/aws-iam-authenticator
      args:
        - "token"
        - "-i"
        - "demo-cluster"

```

Then I override the config file in root -> .kube with the config file I just updated ( my droplet has two config files one in the default root path and another in .kube/ folder)

-> mv config .kube/

```

root@jenkins-server:~# ls
LICENSE  README-zh_CN.md  README.md  config  helmfile.tar.gz
root@jenkins-server:~# vim config
root@jenkins-server:~# ls .kube/config
.kube/config
root@jenkins-server:~# ls .kube
cache  config  config.backup
root@jenkins-server:~# mv config .kube/
root@jenkins-server:~# ls .kube
cache  config  config.backup
root@jenkins-server:~# vim .kube/config
root@jenkins-server:~#

```

My droplet .kube folder is mounted in the Jenkins container so now Jenkins has access to the new configuration and I don't need to create a new container.

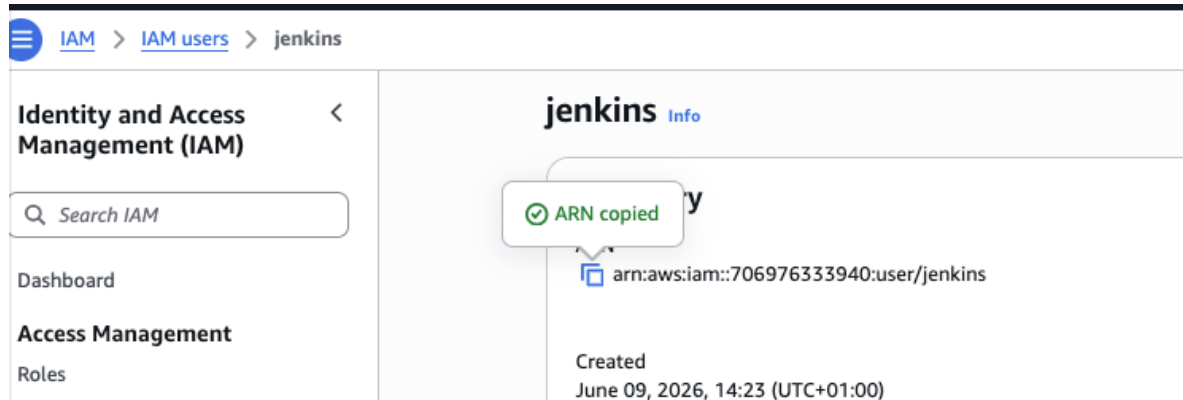
## Sixth attempt failed

Because I created a new EKS cluster — the aws-auth ConfigMap starts fresh with only the node group role.

The Jenkins IAM user entry I added before is gone.

```
[Pipeline] {
[Pipeline] echo
deploying docker image...
[Pipeline] sh
+ envsubst
+ kubectl apply -f -
error: error validating "STDIN": error validating data: failed to download openapi: the server has asked for the client to provide credentials; if you choose to ignore these errors, turn validation off with --validate=false
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // withEnv
```

So I need to edit was-auth again, so I get the Jenkins user ARN:



And from my laptop I execute:

-> kubectl edit configmap aws-auth -n kube-system

And update the rolearn attribute:

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
data:
  mapRoles: |
    - rolearn: arn:aws:iam::706976333940:user/jenkins
      groups:
        - system:bootstrappers
        - system:nodes
      username: system:node:{{EC2PrivateDNSName}}
kind: ConfigMap
metadata:
  creationTimestamp: "2026-06-11T10:12:02Z"
  name: aws-auth
  namespace: kube-system
  resourceVersion: "1237"
  uid: e9c6ef8b-e8b3-4c8d-9dd8-be7c8dc98378
~
```

And is done:

```
astrid@192.168.1.112 /Users/astrid
% kubectl edit configmap aws-auth -n kube-system
configmap/aws-auth edited
astrid@192.168.1.112 /Users/astrid
% █
```

## Seventh attempt failed

Because the edition to was-auth was wrong:

```
+ envsubst
+ kubectl apply -f -
error: error validating "STDIN": error validating data: failed to download openapi: the server has asked for the client to provide credentials; if you choose to ignore these
errors, turn validation off with --validate=false
[Pipeline] }
```

I edited mapRoles and I needed to add mapUsers and use the Jenkins user ARN there.

First get the rolearn value back:

```
-> aws eks describe-nodegroup \
    --cluster-name demo-cluster \
    --nodegroup-name demo-nodes \
    --region eu-west-2 \
    --query 'nodegroup.nodeRole'
```

```
astrid@192.168.1.112 /Users/astrid
% aws eks describe-nodegroup \
    --cluster-name demo-cluster \
    --nodegroup-name demo-nodes \
    --region eu-west-2 \
    --query 'nodegroup.nodeRole'
"arn:aws:iam::706976333940:role/eksctl-demo-cluster-nodegroup-demo-NodeInstanceRole-0XrcbhBxLNJD"
astrid@192.168.1.112 /Users/astrid
```

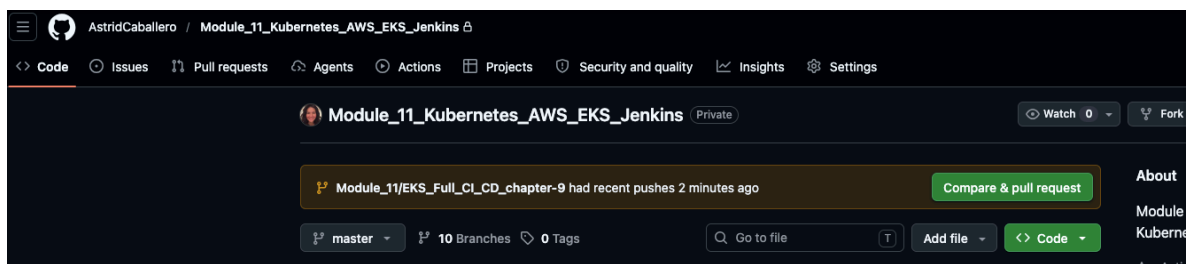
Now edit the mapRoles -> rolearn and add mapUsers and use the Jenkins user ARN in the attribute 'userarn':

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
data:
  mapRoles: |
    - rolearn: arn:aws:iam::706976333940:role/eksctl-demo-cluster-nodegroup-demo-NodeInstanceRole-0XrcbhBxLNJD
      groups:
        - system:bootstrappers
        - system:nodes
      username: system:node:{{EC2PrivateDNSName}}
  mapUsers: |
    - userarn: arn:aws:iam::706976333940:user/jenkins
      username: jenkins
      groups:
        - system:masters
kind: ConfigMap
metadata:
  creationTimestamp: "2026-06-11T10:12:02Z"
  name: aws-auth
  namespace: kube-system
  resourceVersion: "32602"
  uid: e9c6ef8b-e8b3-4c8d-9dd8-be7c8dc98378
~
~
~
~
~
~
```

Eighth attempt passed 🥳❤️



Jenkins pushed changes:



Module\_11/EKS\_Full\_CI\_CD\_chapter-9 had recent pushes 3 minutes ago [Compare & pull request](#)

Module\_11/EKS\_Full\_CI\_CD\_c... 10 Branches 0 Tags

This branch is 140 commits ahead of and 14 commits behind master.

| File        | Commit Message                       | Time Ago      |
|-------------|--------------------------------------|---------------|
| kubernetes  | [Module_11] Add kubectl deploy stage | 2 hours ago   |
| src         | Updated dependency and application   | 4 months ago  |
| target      | ci: version bump                     | 3 minutes ago |
| .gitignore  | Update .gitignore                    | 3 years ago   |
| Dockerfile  | Updated dependencies                 | 4 months ago  |
| Jenkinsfile | edit maven config                    | 2 hours ago   |
| pom.xml     | ci: version bump                     | 3 minutes ago |

[README](#)

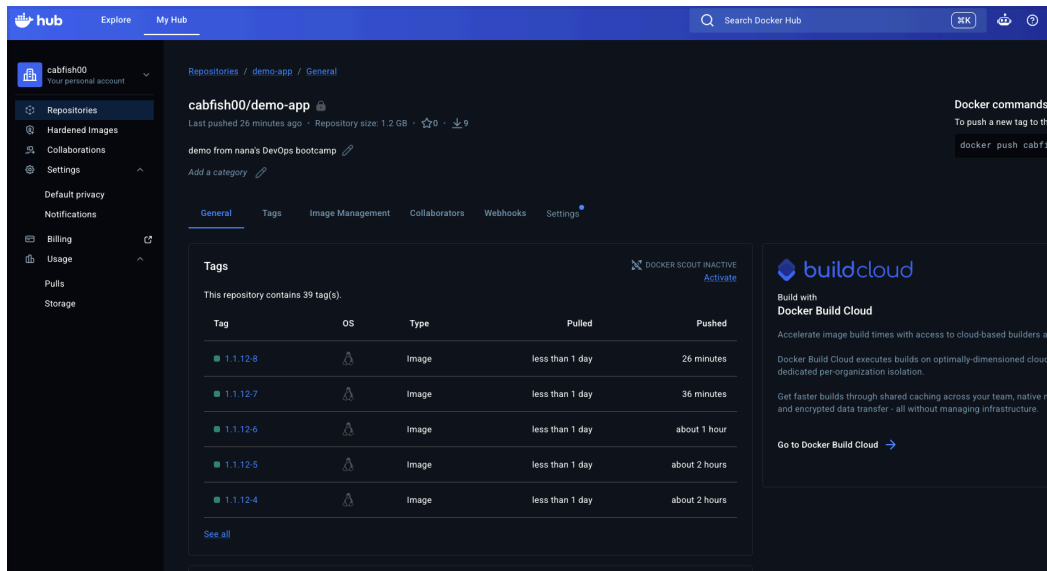
'Build image' stage:

```

--> 45df3705e1a8
Successfully built 45df3705e1a8
Successfully tagged cabfish00/demo-app:1.1.12-8
[Pipeline] sh
+ echo ***
+ docker login -u cabfish00 --password-stdin docker.io
Login Succeeded
[Pipeline] sh
+ docker push cabfish00/demo-app:1.1.12-8
The push refers to repository [docker.io/cabfish00/demo-app]
f20f79f15f59: Preparing
20b7ce5d80fe: Preparing
bc5a3fe779ad: Preparing
20b7ce5d80fe: Layer already exists
bc5a3fe779ad: Layer already exists
f20f79f15f59: Pushed
1.1.12-8: digest: sha256:5849b52479101de6e941f8fdb54786ade9918406849b488ba36aa12cebe64191 size: 953
[Pipeline] }
[Pipeline] // withCredentials

```

I can see all the images created by each Jenkins Pipeline job that I run:



'Deploy' stage:

```
[Pipeline] sh
+ envsubst
+ kubectl apply -f -
deployment.apps/java-maven-app created
[Pipeline] sh
+ envsubst
+ kubectl apply -f -
service/java-maven-app created
[Pipeline] }
[Pipeline] // script
[Pipeline] }
```

The deployment.yaml file test K8s to pull the image:

```
Jenkinsfile x deployment.yaml x service.yaml x Dockerfile x
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: $APP_NAME
5    labels:
6      app: $APP_NAME
7  spec:
8    replicas: 2
9    selector:
10     matchLabels:
11       app: $APP_NAME
12   template:
13     metadata:
14       labels:
15         app: $APP_NAME
16     spec:
17       imagePullSecrets:
18         - name: my-registry-key
19     containers:
20       - name: $APP_NAME
21         image: $DOCKER_REPO:$IMAGE_NAME
22         imagePullPolicy: Always
23         ports:
24           - containerPort: 8080
25
```

'commit version update' stage:

```

[Pipeline] sh
Warning: A secret was passed to "sh" using Groovy String interpolation, which is insecure.
Affected argument(s) used the following variable(s): [PASS]
See https://jenkins.io/redirect/groovy-string-interpolation for details.
+ git remote set-url origin https://AstridCaballero:\*\*\*@github.com/AstridCaballero/Module\_11\_Kubernetes\_AWS\_EKS\_Jenkins.git
[Pipeline] sh
+ git add .
[Pipeline] sh
+ git commit -m ci: version bump
[detached HEAD bceba85] ci: version bump
8 files changed, 25 insertions(+), 28 deletions(-)
[Pipeline] sh
+ git push origin HEAD:Module_11/EKS_Full_CI_CD_chapter-9
To https://github.com/AstridCaballero/Module\_11\_Kubernetes\_AWS\_EKS\_Jenkins.git
4e78bfb..bceba85 HEAD -> Module_11/EKS_Full_CI_CD_chapter-9
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] // node
[Pipeline] End of Pipeline

GitHub has been notified of this commit's build result

Finished: SUCCESS

```

And check the pods:

-> kubectl get pod

And two replicas for the java-maven-app are running 

```

% kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
java-maven-app-757cd65884-vjrbv    0/1     Pending   0           5m24s
java-maven-app-757cd65884-wh8jx    0/1     Pending   0           5m24s
astrid@192.168.1.112 /Users/astrid
%

```

-> kubectl **describe** pod java-maven-app-757cd65884-vjrbv

```

astrid@192.168.1.112 /Users/astrid
% kubectl describe pod java-maven-app-757cd65884-vjrbv
Name:          java-maven-app-757cd65884-vjrbv
Namespace:     default
Priority:       0
Service Account: default
Node:          <none>
Labels:        app=java-maven-app
               pod-template-hash=757cd65884
Annotations:   <none>
Status:        Pending
IP:            <none>
IPs:           <none>
Controlled By: ReplicaSet/java-maven-app-757cd65884
Containers:
  java-maven-app:
    Image:      cabfish00/demo-app:1.1.12-8
    Port:       8080/TCP
    Host Port:  0/TCP
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-8c5rr (ro)
Conditions:
  Type             Status
  PodScheduled     False
Volumes:
  kube-api-access-8c5rr:
    Type:          Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:  kube-root-ca.crt
    ConfigMapOptional: <nil>
    DownwardAPI:    true
QoS Class:       BestEffort
Node-Selectors:  <none>
Tolerations:     node.kubernetes.io/not-ready:NoExecute op=Exists for 300s

```

Also we can see the deployment:

-> kubectl get deployment

```

astrid@192.168.1.112 /Users/astrid
% kubectl get deployment
NAME                READY    UP-TO-DATE    AVAILABLE    AGE
java-maven-app      0/2      2             0            27m
astrid@192.168.1.112 /Users/astrid
%

```

And the service:

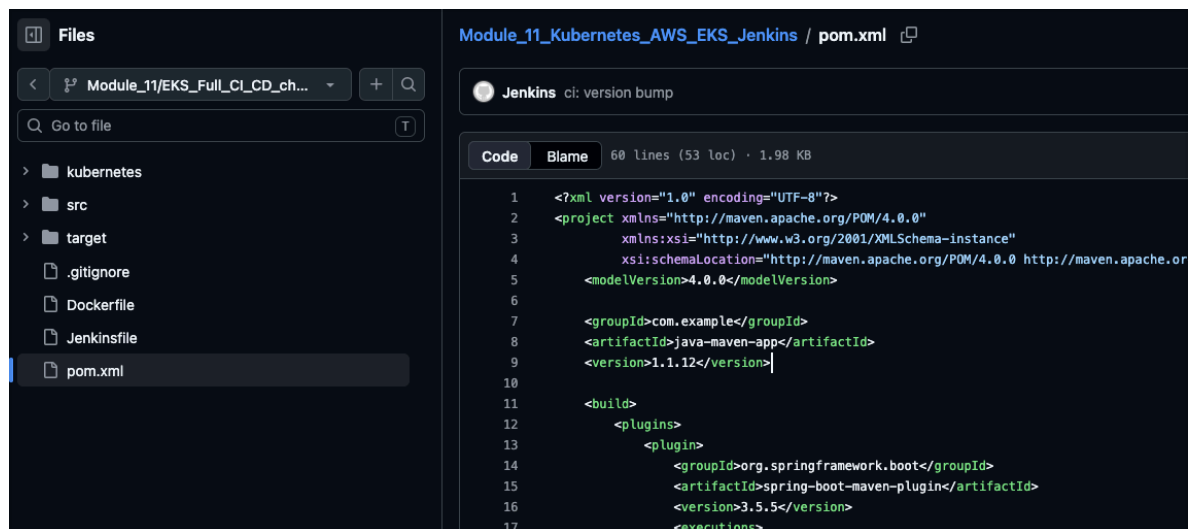
-> kubectl get service

```

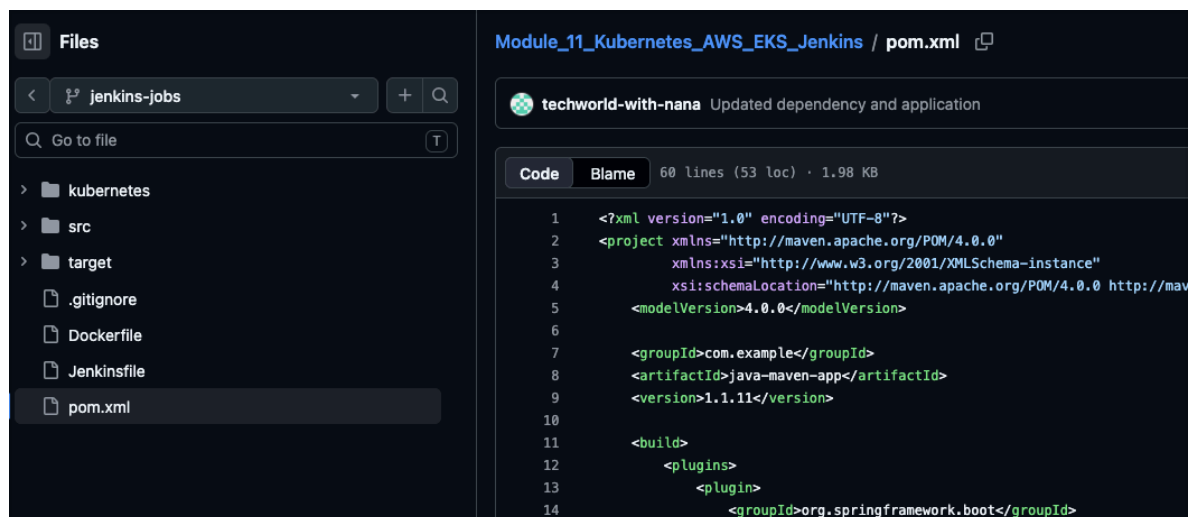
astrid@192.168.1.112 /Users/astrid
% kubectl get service
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
java-maven-app      ClusterIP   10.100.32.21  <none>          80/TCP     28m
kubernetes          ClusterIP   10.100.0.1    <none>          443/TCP    3h53m
astrid@192.168.1.112 /Users/astrid
%

```

And version was updated to 1.1.12:



Version was 1.1.11 (from branch Jenkins-jobs, my working branch is a branch of Jenkins-jobs):



**Jenkins** Search (⌘+K) nana log out

Dashboard > java-maven-app > jenkins-jobs >

**Pipeline jenkins-jobs**

Full project name: microservice-user-auth/jenkins-jobs

**Stage View**

| Stage                                              | Declarative: Checkout SCM | Declarative: Tool Install | increment version | build app | build image | deploy | commit version update |
|----------------------------------------------------|---------------------------|---------------------------|-------------------|-----------|-------------|--------|-----------------------|
| Average stage times: (Average full run time: ~35s) | 4s                        | 83ms                      | 4s                | 7s        | 11s         | 2s     | 2s                    |
| Build History                                      | 4s                        | 83ms                      | 4s                | 7s        | 11s         | 2s     | 2s                    |

Build History: 11 Oct 2023, 09:16

Atom feed for all Atom feed for failures

Increment version -> Build App -> Build & push to Docker Repo -> Deploy to K8s cluster -> Commit version update

REST API Jenkins 2.401.1

I delete the EKS cluster at the end of the lab:  
-> eksctl delete cluster --name demo-cluster